Graph Algorithms for Cybersecurity

Attack-Path Analysis, Critical-Asset Identification
and Anomaly Detection

Marwadi University

Faculty of Engineering and Technology

Department of Computer Science and Engineering

A Research Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology (B.Tech)

In

Computer Science and Engineering

By

AKSHAY BADAMPUDI (92400118531)

Division: 5EN14

Under the Guidance of

Prof. Abhishek Chauhan

Assistant Professor

Department of Computer Science and Engineering

October 2026

Graph Algorithms for Cybersecurity

Attack-Path Analysis, Critical-Asset Identification and Anomaly Detection

Akshay Badampudi (92400118531), Division 5EN14
Department of Computer Science and Engineering, Faculty of Engineering and Technology
Marwadi University, Rajkot, Gujarat, India
Guide: Prof. Abhishek Chauhan, Assistant Professor

Abstract—Computer networks, software dependencies, system-call traces and attacker behaviour are all naturally graph-structured, which makes graph algorithms a core toolbox for cybersecurity. This paper surveys how classical graph algorithms (BFS/DFS, Dijkstra, A*, centrality, max-flow/min-cut, k-shortest paths, community detection) and graph neural networks are applied to attack-graph analysis, provenance-based intrusion detection and botnet discovery. We then formulate attack-path analysis as a shortest-path problem over log-probability weights, adapt a weighted A* search whose heuristic weight trades optimality for speed, and propose *attack-path betweenness*, a K-shortest-path centrality for choosing which hosts to harden. On a synthetic 300-host, 1754-edge enterprise network, weighted A* (ω = 4) expands 64% fewer nodes than Dijkstra, and on 1,600-host graphs weighted A* (ω = 2) expands 41% fewer nodes. Hardening 40 hosts chosen by attack-path betweenness lowers the attacker's best success probability from 47.5% to 15.6%, outperforming PageRank and degree rankings, while community detection with a server-contact feature separates a simulated botnet from benign dense groups. Limitations and future directions are discussed.

Index Terms*—Cybersecurity, graph algorithms, attack graph, A* algorithm, centrality, community detection, provenance graph, graph neural network.*

I. INTRODUCTION

Modern organisations run thousands of interconnected hosts, containers, cloud services and IoT devices. Nearly every question a security analyst asks is a question about relationships: which host can reach which, which vulnerability enables which next step, which process spawned which file write, which machines talk to the same command-and-control server. These relationships form graphs, and graph theory supplies both a language to describe them and a mature set of algorithms to reason about them.

Graph-based security analysis has produced three major research lines. The first is *attack-graph analysis*, which enumerates how an adversary can chain exploits to reach a critical asset [1]–[3]; surveys of generation and analysis techniques are given in [4]–[6]. The second is *provenance-based detection and forensics*, in which operating-system events are stored as a causal graph that analysts traverse to reconstruct an intrusion [8]–[13]. The third is *graph representation learning*, where graph neural networks (GNNs) learn structural signatures of malicious behaviour [14]–[19].

Despite this breadth, the underlying algorithms are usually presented inside larger systems, which makes it hard for a student or practitioner to see which classical algorithm solves which security task and what it costs. This paper makes the following contributions:

- A task-to-algorithm taxonomy that maps security problems to graph algorithms with their complexity (Section III).
- A formulation of attack-path analysis as shortest-path search over log-probability edge weights, with a weighted A* variant whose heuristic weight trades optimality for speed, motivated by heuristic-weighting ideas used in robot path planning [28] (Section IV).
- *Attack-path betweenness*, a centrality computed over the K most likely attack paths, used to choose hosts to harden (Section IV).
- A reproducible simulation study comparing algorithms on attack-path search, hardening, botnet detection and provenance backtracking (Sections V–VI).

The rest of the paper is organised as follows. Section II reviews graph models and related work, Section III describes the algorithms, Section IV presents the framework, Sections V and VI give the experimental set-up and results, Section VII discusses challenges, Section VIII outlines future directions and Section IX concludes.

II. BACKGROUND AND RELATED WORK

A. Graph Models in Security

Different security problems call for different graphs (Table I). A *reachability graph* records which hosts can communicate under the firewall policy. An *attack graph* adds vulnerabilities: nodes are hosts or attacker privilege states and edges are exploit steps, optionally weighted by success probability or cost [1], [3]. *Attack trees* decompose an attacker goal into sub-goals with AND/OR nodes [7]. A *provenance graph* is a directed acyclic graph of system entities (processes, files, sockets) and the events between them [8], [9]. A *flow graph* connects IP addresses that exchange traffic and is the usual input for botnet detection.

TABLE I
GRAPH MODELS USED IN CYBERSECURITY

Graph model	Nodes	Edges	Typical tasks
Reachability	Hosts, subnets	Allowed connections	Blast radius, segmentation
Attack graph	Hosts / privilege states	Exploit steps (prob., cost)	Attack paths, hardening
Attack tree	Goals, sub-goals	AND / OR refinement	Threat modelling
Provenance	Processes, files, sockets	Causal events	APT detection, forensics
Flow graph	IP addresses	Observed traffic	Botnet, anomaly detection

B. Attack Graphs

Sheyner et al. generated attack graphs by model checking a network model against a security property [1]. Ammann et al. made analysis scalable by adopting a monotonicity assumption, so that an attacker never gives up a privilege once gained [3]. MulVAL expresses vulnerabilities and network configuration as logic facts and derives attack paths by Datalog reasoning [2]. Barik et al. consolidate generation and analysis techniques, including alert correlation, network hardening and security metrics [4]; Shandilya et al. critically review attack-graph use in security systems [5]; and Konsta et al. survey the automatic generation of attack trees and attack graphs [6]. The consistent open problem is scalability, because the number of states grows quickly with network size.

C. Provenance-Based Detection

King and Chen introduced backtracking from a detection point to the intrusion's entry point over dependency graphs [10]. HOLMES correlates suspicious information flows into a high-level graph aligned with the stages of an advanced persistent threat (APT) [11]. UNICORN builds compact summaries of streaming provenance graphs to learn normal behaviour at runtime [12], and NoDoze uses provenance to triage alerts and reduce alert fatigue [13]. Surveys document the strengths of provenance graphs for APT detection and the challenges of data volume, dependency explosion and benchmark datasets [8], [9].

D. Graph Neural Networks

Surveys of GNN-based intrusion detection review graph construction, model design, datasets and evaluation [14]–[16]. Common architectures include graph convolutional networks [17], GraphSAGE [18] and graph attention networks [19]. GNNs capture structural patterns that flat feature vectors miss, but the surveys also report limited labelled data and weak robustness to evasion as open issues. Non-graph baselines such as Kitsune [20] remain strong for online detection.

E. Comparison of Representative Approaches

Table II summarises how the surveyed systems differ in graph type, core technique and limitations; the last row positions the present study.

TABLE II
COMPARISON OF REPRESENTATIVE APPROACHES

Approach	Graph	Core technique	Main limitation
MulVAL [2]	Attack graph	Datalog reasoning	Needs accurate vulnerability data
Ammann et al. [3]	Attack graph	Monotonic reachability	Ignores privilege loss
HOLMES [11]	Provenance	Flow correlation to APT stages	Relies on predefined behaviour patterns
UNICORN [12]	Provenance	Streaming graph summaries, learning	Sensitive to benign behaviour drift
NoDoze [13]	Provenance	Anomaly score propagation	Needs historical profile
GNN IDS [14]–[16]	Flow / provenance	GCN, GAT, graph autoencoder	Scarce labels, weak robustness
This study	Attack + flow	A*, Yen, centrality, Louvain	Synthetic validation only

F. Research Gap and Objectives

Existing surveys describe systems at a high level, and system papers evaluate full pipelines, so the individual contribution of each classical algorithm is rarely isolated. The objectives of this study are therefore: (1) to relate each graph algorithm to a concrete security task; (2) to measure search cost and solution quality of Dijkstra and A* variants on probabilistic attack graphs; (3) to compare node-ranking strategies for patch prioritisation; and (4) to test community detection for botnet discovery and depth-limited traversal for provenance triage, all under controlled, reproducible conditions.

III. GRAPH ALGORITHMS AND THEIR SECURITY ROLES

A. Traversal: BFS and DFS

Breadth-first search (BFS) and depth-first search (DFS) run in O(V+E) [23]. In security, BFS from a compromised host yields the *blast radius* (all hosts reachable) and the minimum number of hops to a crown-jewel asset. Backward BFS over a provenance graph returns every ancestor of an alert, which is the basis of backtracking [10].

*B. Shortest Paths: Dijkstra and A**

Dijkstra's algorithm [21] finds minimum-cost paths in O((V+E) log V) with a binary heap. A* [22] adds a heuristic h(n) to the evaluation function and expands the node with the lowest f(n):

$$f(n) = g(n) + \omega \cdot h(n) \tag{1}$$

where g(n) is the cost from the source and $\omega \geq 1$ is a heuristic weight. If h never overestimates the true remaining cost and $\omega = 1$, A* returns an optimal path. For $\omega > 1$ the search is greedier and expands fewer nodes, and the returned path costs at most ω times the optimum. This speed/quality trade-off is the same one reported when heuristic weights and parent-node terms are added to A* for mobile-robot path planning [28].

C. Centrality

Degree, betweenness [25] and PageRank [24] rank nodes by importance. Betweenness counts the fraction of shortest paths between distinct nodes s and t that pass

through a node v,

$$C_B(v) = \Sigma_{s,t} \sigma_{st}(v) / \sigma_{st} \quad (2)$$

and PageRank gives the stationary visit probability of a random walker with damping d:

$$PR(v) = (1-d)/N + d \Sigma_{u \to v} PR(u)/L(u) \quad (3)$$

where L(u) is the out-degree of u. Betweenness highlights pivot hosts that sit between many pairs; PageRank highlights assets that many paths lead to.

D. Max-Flow/Min-Cut and k-Shortest Paths

By the max-flow min-cut theorem [29], the smallest set of edges whose removal separates attackers from targets equals the maximum flow, giving the minimum number of firewall rules or patches needed for full isolation. Yen's algorithm [26] enumerates the K lowest-cost loopless paths, which exposes alternative attack routes rather than a single best one.

E. Community Detection

Communities are node groups that are densely connected internally. The Louvain method [27] greedily maximises modularity [30],

$$Q = (1/2m) \Sigma_{ij} [A_{ij} - k_i k_j / 2m] \delta(c_i, c_j) \quad (4)$$

where A is the adjacency matrix, k_i the degree of node i and m the number of edges. Peer-to-peer botnets form tightly meshed communities in flow graphs, making community structure a natural detection signal.

F. Graph Neural Networks

A graph convolutional layer [17] updates node features by aggregating neighbours:

$$H^{(l+1)} = \sigma(D'^{-1/2} A' D'^{-1/2} H^{(l)} W^{(l)}) \quad (5)$$

with $A' = A + I$, D' its degree matrix and $W^{(l)}$ trainable weights. Stacked layers let each node absorb information from multi-hop neighbourhoods, which is useful for spotting multi-step attacks [14].

G. Subgraph Matching and Graph Similarity

Detecting a known attack pattern in a provenance graph is a subgraph-isomorphism problem, which is NP-complete in general, so practical systems restrict the pattern size, index node labels and prune candidates early; HOLMES-style correlation of suspicious flows is one way to keep matching tractable [11]. To compare whole graphs, for example two malware call graphs or two daily communication graphs, systems use graph edit distance, graph kernels or iterative label-hashing (Weisfeiler–Lehman) to turn each graph into a fixed-length signature that can be compared quickly; streaming sketches of this kind underpin runtime detectors such as UNICORN [12].

H. Application Domains

- **Cloud and identity.** Permissions, roles and trust relationships form a directed graph; shortest-path search from a low-privilege account to an administrator role reveals privilege-escalation chains.
- **Software supply chain.** A dependency graph with BFS from a vulnerable library identifies every product that transitively includes it and the shortest route by which it is reachable.
- **Malware analysis.** Function call graphs and control-flow graphs are compared with graph similarity to group variants of the same family.
- **Industrial and IoT networks.** Physical connectivity and protocol graphs let defenders compute which devices an attacker can reach from a compromised gateway and where to place monitors.
- **Fraud and phishing.** Bipartite graphs of domains and IP addresses, or accounts and devices, expose clusters of related malicious infrastructure through connected components and community detection.

TABLE III
SECURITY TASKS AND THEIR GRAPH ALGORITHMS

Security task	Algorithm	Complexity	Output
Blast radius	BFS / DFS	O(V+E)	Reachable set
Most likely attack path	Dijkstra, A*	O((V+E) log V)	Path, probability
Alternative routes	Yen's K-shortest	O(KV(E+V log V))	K paths
Critical assets	PageRank	O(k(V+E))	Importance rank
Choke points	Betweenness	O(VE)	Pivot hosts
Isolation cost	Max-flow / min-cut	poly.	Min edge cut
Botnet discovery	Louvain	≈ O(E log V)	Communities
Alert triage	Backward BFS	O(V+E)	Causal ancestors

IV. PROPOSED ANALYSIS FRAMEWORK

A. Pipeline

Fig. 1 shows the framework. Logs, flow records and vulnerability scans are converted into a graph, pruned and weighted, analysed by the algorithm layer, and turned into a ranked risk list that drives response actions such as patching, isolation or alerting.

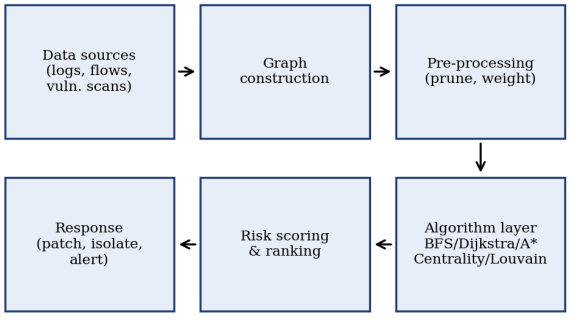


Fig. 1. Graph-based security analysis pipeline.

B. Probabilistic Attack Graph

Each exploit edge (u,v) has a success probability p(u,v). Because probabilities multiply along a path, we use $w(u,v) = -\ln p(u,v)$ so that the most likely attack path is the minimum-weight path:

$$P(\pi) = \Pi p(u,v) = exp(-\Sigma w(u,v)) \quad (6)$$

Dijkstra, A* and Yen's algorithm can then be applied unchanged.

C. Weighted A* for Layered Networks

Enterprise networks are segmented into zones (DMZ, user, application, database, control). Let $\lambda(n)$ be the number of zone layers still to be crossed from n to the goal zone, and let j_{max} be the largest layer jump any single edge can make. Then $h(n) = \text{ceil}(\lambda(n)/j_{max})\cdot w_{min}$ never overestimates the remaining cost, because at least $\text{ceil}(\lambda/j_{max})$ edges must be traversed and each costs at least w_{min}. In our topology $j_{max} = 2$. Setting $\omega > 1$ in Eq. (1) gives faster, bounded-suboptimal search.

Algorithm 1: Weighted A* attack-path search

Input: graph G, source S, target T, weight ω
1. open ← {S}; g(S) ← 0; closed ← empty set
2. **while** open is not empty **do**
3. n ← node in open with least f = g + ω·h
4. **if** n = T **then return** path(n)
5. move n to closed
6. **for** each successor m of n **do**
7. **if** m in closed **continue**
8. **if** g(n)+w(n,m) < g(m) **then** update g(m), parent(m); push m
9. **return** no path

D. Attack-Path Betweenness

Standard betweenness (Eq. 2) averages over *all* node pairs, most of which are irrelevant to an attacker. We instead compute centrality only over the K most likely attack paths Π_K between attacker entry points and targets, obtained with Yen's algorithm:

$$C_K(v) = (1/K)\,\Sigma_{\pi\ in\ \Pi_K}\ 1[v \text{ lies on } \pi] \qquad (7)$$

Hosts with the highest C_K are hardened first, since they lie on the largest share of probable attack routes.

Algorithm 2: Greedy hardening by attack-path betweenness

Input: graph G, entry hosts S, targets T, budget k, path count K
1. Π ← first K shortest S–T paths (Yen) over w = −ln p
2. **for** each host v: C(v) ← number of paths in Π containing v
3. choose the k removable hosts with largest C(v)
4. patch or isolate them; recompute best path probability
5. **repeat** from step 1 at the next patch cycle

E. Botnet Scoring

After Louvain partitioning of the flow graph, a community of size 8 to 80 is flagged when its internal density exceeds 0.12 *and* less than 12% of its members' edges go to known servers. Benign dense groups (departments, file-sharing groups) talk mostly to servers, whereas bots mesh with each other and contact a command server rarely.

F. Correctness and Complexity

Proposition 1 (admissibility). The heuristic $h(n) = \text{ceil}(\lambda(n)/j_{max})\cdot w_{min}$ never exceeds the true remaining cost h*(n). *Proof.* Any path from n to the goal zone crosses $\lambda(n)$ layers and each edge advances at most j_{max} layers, so the path has at least $\text{ceil}(\lambda(n)/j_{max})$ edges, each costing at least w_{min}. Hence A* with $\omega = 1$ returns an optimal path [22].

Proposition 2 (bounded suboptimality). With $\omega \geq 1$ and an admissible h, the path cost returned is at most $\omega\cdot C^*$, where C* is the optimal cost. *Proof.* Just before the goal is selected, some node n′ on an optimal path is in the open list with $g(n') = g^*(n')$. Then $f(n') = g^*(n') + \omega\cdot h(n') \leq \omega(g^*(n') + h^*(n')) = \omega C^*$. The goal is selected only if its f-value is no larger than f(n′), so its cost is at most ωC*. In probability terms, the path found has success probability at least $(P^*)^{\omega}$, where P* is the optimal probability.

Complexity. With a binary heap, Algorithm 1 runs in O(E log V) in the worst case and in practice expands only a fraction of the nodes. Attack-path betweenness costs one run of Yen's algorithm, O(KV(E + V log V)), plus O(K·L) to count node occurrences on paths of length L; greedy hardening of k hosts repeats a shortest-path query after every removal, adding O(k(E + V log V)).

G. Worked Example

Fig. 2 shows a seven-node attack graph; edge labels are exploit success probabilities. Dijkstra on w = −ln p settles nodes in the order of Table IV. The most likely route to the domain controller is Internet → Web → App → DB → DC with success probability 0.3024 (product 0.8 × 0.7 × 0.6 × 0.9). The alternative route through the VPN and workstation has a lower probability (0.5 × 0.9 × 0.6 × 0.6 × 0.9 = 0.1458), so patching the web-server exploit would force the attacker onto it and cut the best success probability by half.

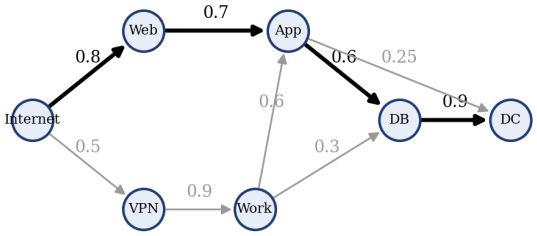


Fig. 2. Worked example: probabilistic attack graph with the most likely path in bold.

TABLE IV
DIJKSTRA TRACE ON THE WORKED EXAMPLE

Settled order	Node	Cost g = −ln P	Probability P
1	Internet	0.000	1.0000
2	Web	0.223	0.8000
3	App	0.580	0.5600
4	VPN	0.693	0.5000
5	Work	0.799	0.4500
6	DB	1.091	0.3360
7	DC	1.196	0.3024

TABLE V
NOTATION USED IN THE FRAMEWORK

Symbol	Meaning
G = (V, E)	Directed graph of hosts / entities and edges
p(u,v), w(u,v)	Exploit success probability; weight −ln p(u,v)
g(n), h(n)	Cost from source to n; heuristic remaining cost
ω, a	Heuristic weight; parent-node weight
$\lambda(n)$, j_{max}	Zone layers left to goal; maximum layer jump per edge
K, Π_K	Number of paths; set of K most likely attack paths
$C_K(v)$	Attack-path betweenness of host v
k	Hardening budget (hosts patched or isolated)
σ	Standard deviation of probability noise

V. EXPERIMENTAL SET-UP

A. Threat Model and Metrics

We assume a network-level adversary who has a foothold on one of three DMZ hosts, knows the topology and chooses the exploit chain that maximises the overall success probability. Exploit attempts on different edges are treated as independent, so path probability is the product of edge probabilities (Eq. 6). The defender can patch or isolate a bounded number k of hosts in the user, application and database zones; DMZ and control hosts are not removable because they are the entry points and the assets being protected. We report four metrics: (i) *nodes expanded* and wall-clock time for search cost; (ii) *best attack success probability* for residual risk; (iii) *targets still reachable* for connectivity; and (iv) precision, recall and F1 for botnet detection.

B. Datasets and Implementation

All experiments use Python 3.12 and NetworkX 3.6.1 [31] on a single CPU core; timings are therefore indicative and should be compared relatively. Every random generator is seeded for reproducibility.

- **Attack network.** 300 hosts in five zones (DMZ 30, user 150, application 75, database 30, control 15) and 1754 directed exploit edges with success probability p ~ U(0.15, 0.95). The attacker enters through three DMZ hosts; the 15 control hosts are targets.
- **Scalability.** Same generator at 100 to 1,600 hosts, averaged over 10 random graphs per size.
- **Botnet flow graph.** 400 hosts including 20 servers and 30 bots. Normal hosts contact three servers and two peers; four benign groups of 25 hosts are densely connected (p = 0.25). Each bot meshes with five other bots, contacts one server and makes three noisy contacts to normal hosts. Results are averaged over 10 seeds.
- **Provenance graph.** A 3,000-node DAG with 5,997 edges, where each event depends on up to two of the previous 60 events; the alert is raised at node 2,990.

Reproducibility. Graph generators, random seeds and algorithm implementations are deterministic given the stated parameters, so every table in Section VI can be regenerated by re-running the scripts with the same NetworkX version. Timing columns depend on the machine and are meaningful only as ratios between methods.

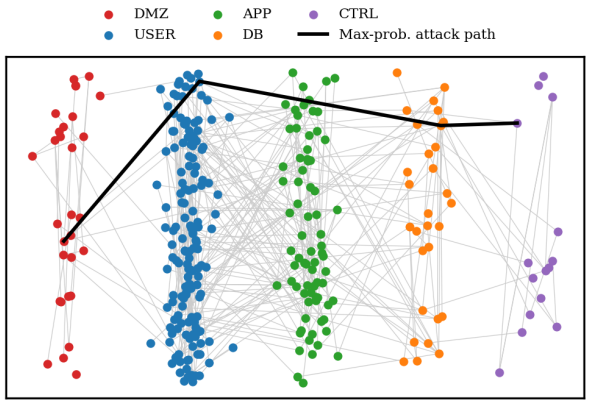


Fig. 3. Synthetic five-zone attack network with the most likely attack path highlighted.

VI. RESULTS AND ANALYSIS

A. Attack-Path Search

In the 300-host network, 95.7% of hosts are reachable from the DMZ foothold, and the shortest route to a control host needs 3 hops. The most likely path succeeds with probability 47.6%. Table VI compares search strategies.

TABLE VI
ATTACK-PATH SEARCH ON THE 300-HOST NETWORK

Algorithm	Nodes expanded	Time (ms)	Path success prob.	Hops
Dijkstra (h=0)	36	0.275	47.55%	3
A* (w=1.0)	30	0.166	47.55%	3
A* weighted (w=2.0)	24	0.111	47.18%	4
A* weighted (w=4.0)	13	0.061	47.18%	4

The admissible heuristic (ω = 1) keeps the optimal path while expanding 17% fewer nodes than Dijkstra. Raising ω to 4 cuts expansions by 64% and time by 78%, at the cost of a path whose success probability is 47.18% instead of 47.55%, i.e. 0.8% lower in relative terms. As in the robot-planning study [28], a larger heuristic weight buys speed and gives up a small amount of path quality.

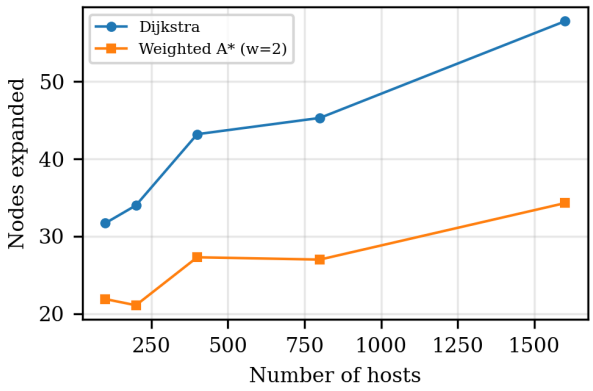


Fig. 4. Nodes expanded versus network size (mean of 10 graphs per size).

Fig. 4 shows that at 1,600 hosts Dijkstra expands 57.8 nodes on average against 34.3 for weighted A* (ω = 2), a 41% reduction, with the gap widening as the network grows. In all runs the paths found had equal cost to Dijkstra's in the generated graphs.

Parent-node variant. The robot-planning study [28] also adds the heuristic of the parent node, f(n) = g(n) + a·[h(n) + h(p)], where p is the node from which n was reached. We implemented this variant and tested it on 30 random 300-host networks (Table VII).

TABLE VII
PARENT-NODE HEURISTIC VARIANT (30 NETWORKS)

Method	Mean nodes expanded	Mean extra cost (%)	Worst extra cost (%)
Dijkstra	39.5	0.00	0.00
A* (w=1)	31.3	0.00	0.00
Parent-node, a=0.5	33.3	0.00	0.00
Parent-node, a=1.0	27.3	0.00	0.00

With a = 1 the parent-node variant expands 31% fewer nodes than Dijkstra and 13% fewer than plain A*, and found optimal paths in all 30 networks. With a = 0.5 it behaves like a damped heuristic and expands more nodes than plain A* (33.3 against 31.3), because halving the heuristic weakens its guidance. Since the parent's heuristic is larger than the child's on layered graphs, the sum h(n) + h(p) can overestimate the remaining cost, so optimality is not guaranteed in general even though no loss was observed here.

Sensitivity to ω. We repeated the search on 30 random 300-host networks for ω between 1 and 6 (Fig. 5). Mean expansion savings against Dijkstra rise from 21% at ω = 1 to 37% at ω = 2, 49% at ω = 4 and 55% at ω = 6. The mean path-cost penalty stays below 0.1% up to ω = 2 (worst case 2.6%) but reaches 2.6% on average and 35% in the worst case at ω = 4. Values of ω between 1.5 and 3 therefore give a good balance on this topology, consistent with the guidance to keep heuristic weights moderate in [28].

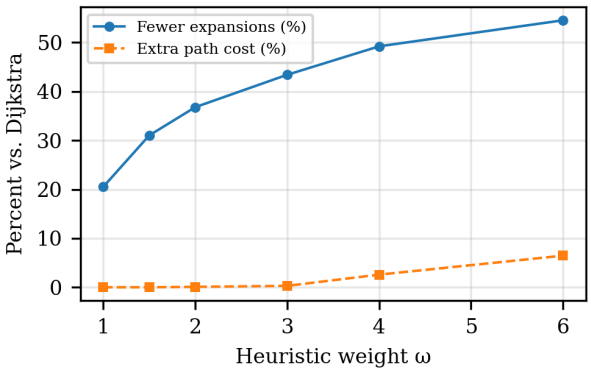


Fig. 5. Effect of the heuristic weight ω (mean of 30 networks).

B. Critical Assets and Hardening

PageRank's top-10 hosts are all control-zone assets, as expected for nodes with many incoming routes, whereas the top-10 by betweenness are all user-zone hosts; the two lists share 0 hosts, confirming that they capture different notions of importance. The minimum edge cut separating the DMZ from the control zone contains 60 edges, so full isolation by cutting links is expensive. Yen's algorithm enumerated K = 200 paths in 0.42 s.

We then removed (patched) k hosts from the user, application and database zones using five ranking strategies and recomputed the attacker's best success probability (Fig. 6, Table VIII).

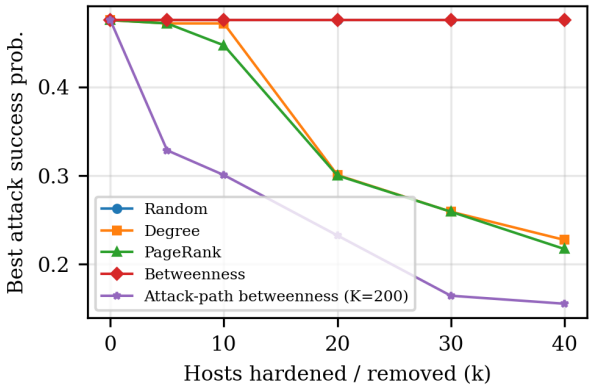


Fig. 6. Best attack success probability after hardening k hosts.

TABLE VIII
BEST ATTACK SUCCESS PROBABILITY (BASELINE 47.5%)

Strategy	k = 10	k = 20	k = 40	Reduction at k = 40
Random	47.5%	47.5%	47.5%	0.0%
Degree	47.2%	30.1%	22.8%	52.1%
PageRank	44.7%	30.0%	21.7%	54.3%
Betweenness	47.5%	47.5%	47.5%	0.0%
Attack-path betw.	30.1%	23.3%	15.6%	67.3%

Attack-path betweenness is the most effective strategy at every budget: ten hosts already reduce the best attack probability by 37%, while PageRank and degree need roughly twenty hosts for a comparable effect. Random selection and standard betweenness leave the best path untouched, because the hosts they select are not on the optimal route. This shows that generic importance measures are poor guides for patching and that centrality must be computed over attack-relevant paths. Note that all 15 targets remain reachable after hardening 40 hosts; node removal lowers attacker success rather than eliminating access, so it should complement segmentation.

C. Botnet Detection

TABLE IX
BOTNET DETECTION ON THE SIMULATED FLOW GRAPH (10 SEEDS)

Method	Precision	Recall	F1
Degree threshold (top 8%)	0.06	0.08	0.07
Louvain + density only	0.36	1.00	0.53
Louvain + density + server ratio	1.00	1.00	1.00

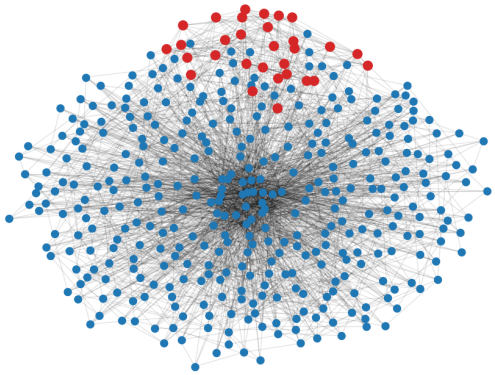


Fig. 7. Simulated flow graph; bots (red) form a compact mesh among normal hosts (blue).

A degree threshold fails because servers and benign group members have high degree. Density alone finds every bot but also flags benign groups (precision 0.36). Adding the server-contact ratio removes the benign groups and gives perfect scores on this synthetic data. We stress that this result reflects the simulator's clear separation between bots and benign groups; real traffic is far noisier, so the numbers demonstrate the method rather than predict field accuracy.

D. Provenance Backtracking

TABLE X
BACKWARD TRAVERSAL FROM ONE ALERT (3,000-NODE GRAPH)

Strategy	Nodes returned	Time (ms)
Full backtrack	2346	2.548
Depth <= 3	15	0.018
Depth <= 5	49	0.021
Depth <= 8	165	0.065

Full backward BFS returns 2,346 ancestors, a typical dependency explosion that no analyst can inspect. Limiting the depth to 5 returns 49 nodes (98% fewer), and depth 3 returns 15. Depth limits trade completeness for readability, which is why systems add anomaly scoring on top of the traversal [13].

E. Robustness to Estimation Error

Exploit probabilities in practice are estimates. To test sensitivity, we multiplied every edge probability by log-normal noise with standard deviation σ, ranked hosts by attack-path betweenness on the *noisy* graph, hardened them, and then measured the best attack probability on the *true* graph (mean of six noise draws). Table XI shows the result.

TABLE XI
HARDENING WITH NOISY PROBABILITY ESTIMATES (BASELINE 47.6%)

Noise σ	k = 20	k = 40	Worst case at k = 40
0.0	23.3%	15.6%	15.6%
0.1	25.6%	17.8%	23.4%
0.2	30.1%	19.0%	23.4%
0.4	40.8%	31.2%	44.7%

With moderate noise (σ = 0.2) hardening 40 hosts still lowers the best attack probability from 47.6% to 19.0%, a lower residual probability than PageRank achieves with exact data (21.7%); even at σ = 0.4 the average residual probability is 31.2%, still about 34% below the baseline. The worst draw at σ = 0.4 (44.7%) is close to the baseline of 47.6%, which indicates that very noisy estimates can mislead the ranking. Defenders should therefore recompute rankings after each patch cycle and combine them with coarse expert judgement when scan data are unreliable.

F. Summary of Findings

- **Search.** An admissible layered heuristic is a free improvement over Dijkstra, and a moderate weight (ω between 1.5 and 3) gives larger savings at negligible loss in path quality on this topology.
- **Prioritisation.** Importance measures answer different questions: PageRank finds assets worth protecting, betweenness finds pivot hosts across all traffic, and attack-path betweenness finds the hosts that matter for the attacker's likely routes. Only the last improved hardening at every budget tested.
- **Detection.** Structure alone (density) is not enough because benign groups are also dense; combining community structure with a simple behavioural feature removed the false positives in simulation.
- **Forensics.** Depth-limited traversal cuts the causal subgraph by about two orders of magnitude and should be paired with scoring, so that analysts see a short, ranked chain instead of thousands of ancestors.

Practical guidance. A defender with limited effort can follow a three-step recipe: run BFS to measure blast radius, run K-shortest-path enumeration from entry points to crown-jewel assets, and patch the hosts that appear on the most paths. After each round of patching the analysis should be repeated, because removing the hosts on the best route moves the attacker to the next best route and changes the ranking.

VII. DISCUSSION AND CHALLENGES

- **Scalability.** Exact attack-graph generation can blow up combinatorially [4], [6]; heuristic search, monotonic assumptions and sampling of K paths are practical remedies.
- **Dynamic data.** Networks and vulnerabilities change daily, so graphs need incremental updates instead of full recomputation.
- **Adversarial evasion.** Attackers can add benign-looking edges or mimic normal communities; robustness of both classical and learned detectors is open [14], [15].
- **Data quality.** Probabilities in attack graphs are estimated (for example from CVSS-style scores) and benchmark provenance datasets are scarce [9].
- **Independence assumption.** Multiplying edge probabilities assumes independent exploit attempts. Shared credentials or a common vulnerable library create correlated edges, which can make real attackers

more successful than the model predicts, so probabilities should be treated as a ranking signal rather than exact risk.

- **Operational cost.** Betweenness is O(VE) and K-shortest-path enumeration grows with K; for large estates, sampling, caching and running analysis per segment are needed to keep results fresh.
- **Validity of this study.** All results use synthetic graphs with seeded generators. They show relative behaviour of algorithms but should be re-run on real scan, flow and audit data before drawing operational conclusions.

Relation to robot path planning. The grid-based planner in [28] searches a 2-D map for a collision-free route, whereas our graph encodes attacker progress through a network. The two settings share the same search machinery: the evaluation function f = g + h, open and closed lists, and a heuristic weight that trades path quality for speed. They differ in the cost being minimised (metric distance versus negative log success probability), in the source of the heuristic (Euclidean or Manhattan distance versus zone layers) and in the meaning of an obstacle (a wall versus a patched host). The results agree qualitatively: larger weights and parent-node terms reduce the number of expanded nodes, while very large weights begin to lose path quality. This suggests that heuristic-search improvements developed for robotics transfer to security graphs once a suitable admissible heuristic is defined.

VIII. FUTURE DIRECTIONS AND ETHICAL USE

- **Real data validation.** Replace the synthetic generators with CVE/CVSS-derived edge probabilities, enterprise flow captures and public provenance benchmarks, and report confidence intervals.
- **Incremental algorithms.** Dynamic shortest-path and streaming community detection can keep results current as hosts, rules and vulnerabilities change.
- **Hybrid classical/learned systems.** Use graph algorithms for fast, explainable candidate selection and GNNs for ranking, and compare them against the classical baselines in [14]–[16].
- **Adversarial robustness.** Evaluate how an attacker who adds decoy edges or imitates benign communities degrades centrality and community-based detection.
- **Ethical use.** Attack-path tools reveal how to compromise a network, so access to their outputs should be restricted, and experiments should run only on systems the analyst is authorised to assess.

IX. CONCLUSION

Graph algorithms give cybersecurity a compact, well-understood toolkit. Our simulations show that an admissible A* heuristic preserves optimality with fewer expansions, that a larger heuristic weight trades a small loss in path quality for much faster search, that attack-path betweenness identifies hosts whose hardening cuts attacker success from 47.5% to 15.6% with 40 patches, and that community structure combined with a server-contact feature isolates botnets in simulation. Future work will validate the framework on real data, add incremental updates for dynamic networks, evaluate GNN detectors against the classical baselines, and study adversarial robustness.

ACKNOWLEDGMENT

The author thanks Prof. Abhishek Chauhan for guidance and the Department of Computer Science and Engineering, Marwadi University, for support.

REFERENCES

[1] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.

[2] X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A logic-based network security analyzer,” in *Proc. USENIX Security Symp.*, 2005, pp. 113–128.

[3] P. Ammann, D. Wijesekera, and S. Kaushik, “Scalable, graph-based network vulnerability analysis,” in *Proc. 9th ACM Conf. Computer and Communications Security*, 2002, pp. 217–224.

[4] M. S. Barik, A. Sengupta, and C. Mazumdar, “Attack graph generation and analysis techniques,” *Defence Sci. J.*, vol. 66, no. 6, pp. 559–567, 2016.

[5] V. Shandilya, C. B. Simmons, and S. Shiva, “Use of attack graphs in security systems,” *J. Comput. Netw. Commun.*, vol. 2014, Art. no. 818957, 2014.

[6] A.-M. Konsta, A. L. Lafuente, B. Spiga, and N. Dragoni, “Survey: Automatic generation of attack trees and attack graphs,” *Comput. Secur.*, vol. 137, Art. no. 103602, 2024.

[7] B. Schneier, “Attack trees,” *Dr. Dobb’s J.*, vol. 24, no. 12, pp. 21–29, 1999.

[8] Z. Li, Q. A. Chen, R. Yang, Y. Chen, and W. Ruan, “Threat detection and investigation with system-level provenance graphs: A survey,” *Comput. Secur.*, vol. 106, Art. no. 102282, 2021.

[9] M. Zipperle, F. Gottwalt, E. Chang, and T. Dillon, “Provenance-based intrusion detection systems: A survey,” *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–36, 2023.

[10] S. T. King and P. M. Chen, “Backtracking intrusions,” in *Proc. 19th ACM Symp. Operating Systems Principles*, 2003, pp. 223–236.

[11] S. M. Milajerdi *et al.*, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in *Proc. IEEE Symp. Security and Privacy*, 2019, pp. 1137–1152.

[12] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “UNICORN: Runtime provenance-based detector for advanced persistent threats,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2020.

[13] W. U. Hassan *et al.*, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. NDSS*, 2019.

[14] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui, “Graph neural networks for intrusion detection: A survey,” *IEEE Access*, vol. 11, pp. 49114–49139, 2023.

[15] M. Zhong, M. Lin, C. Zhang, and Z. Xu, “A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges,” *Comput. Secur.*, vol. 141, Art. no. 103821, 2024.

[16] E. Yagmur, C. Koçak, and A. Keles, “Graph neural networks and graph autoencoders for intrusion detection systems: A systematic review, taxonomy and comparative analysis,” *Comput. Netw.*, vol. 288, Art. no. 112598, 2026.

[17] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[18] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1024–1034.

[19] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.

[20] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. NDSS*, 2018.

[21] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[22] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[23] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[24] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the Web,” Stanford InfoLab, Tech. Rep., 1999.

[25] L. C. Freeman, “A set of measures of centrality based on betweenness,” *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.

[26] J. Y. Yen, “Finding the *k* shortest loopless paths in a network,” *Manage. Sci.*, vol. 17, no. 11, pp. 712–716, 1971.

[27] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *J. Stat. Mech.*, vol. 2008, no. 10, P10008, 2008.

[28] “A mobile robot path planning algorithm based on improved A* algorithm,” course reference paper supplied by the guide, Marwadi Univ., 2026.

[29] L. R. Ford Jr. and D. R. Fulkerson, “Maximal flow through a network,” *Can. J. Math.*, vol. 8, pp. 399–404, 1956.

[30] M. E. J. Newman, “Modularity and community structure in networks,” *Proc. Natl. Acad. Sci. USA*, vol. 103, no. 23, pp. 8577–8582, 2006.

[31] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.